Two Models, One Compiler: Verifier-Centric Coordination for Competition Mathematics in Lean 4

Jacob (jr4682@columbia.edu)
Columbia University

Preprint · Verified Mechanisms take-home submission · August 30, 2026

Abstract

Problem. Two fixed models, `qwen/qwen3.5-flash-02-23` and `openai/gpt-oss-120b`, must jointly solve competition-mathematics problems and prove the results in Lean 4 under per-problem caps of $1.00 and 8 hours, scored by a kernel-level Comparator. **Method.** We build a verifier-centric staged controller: deterministic tactics, pooled cross-model sampling, shallow compiler-guided repair, cross-model handoff on repair plateaus, sketch-and-fill decomposition, and a *comparator precheck* that re-checks accepted proofs with the scoring Comparator itself instead of trusting the faster development checker alone. **Main controlled result.** At identical 30-minute caps, post-review reruns of both raw baselines reach aggregate parity with the controller's arms (raw Qwen 9/16, raw GPT-OSS 10/16, arms 8–10/16). In fact, most of the controller's earlier apparent gain over the shorter-cap kit baselines was extra wall-clock. The two-model system reaches 9–10/16 with complementary per-problem outcomes across seeds, and the controller's measured value lies in zero-cost deterministic coverage of the easy tier, complementarity, robustness, and verifier-alignment machinery, not in aggregate lift over a cap-matched raw loop. The aggregate gap (+1) sits within our observed run-to-run variation, so we treat per-problem complementarity, not the aggregate, as the primary evidence. On our held-out custom problems the scaffold does separate from the raw loop (duo 7/8, solo-Qwen 6/8, raw Qwen 4/8 at matched caps); the extra duo solve traces to a Qwen sampling wave, not to model interaction. **Benchmark audit.** Writing reference proofs surfaced two defective kit statements (one false as formalized, one unscorable under the gate's imports). An upstream revision then corrected the false bound to exactly the witness value in our certificate and closed a definitional shortcut that every historical Putnam pass had used. **Hard instances.** Of the four provable hard problems missed by both original raw baselines, one fell within 30-minute caps (to a controller arm and, once, to the raw GPT-OSS rerun); multi-hour decomposition runs solved two more; the fourth remains open. A single-model arm given the same controller and a long horizon reproduced one of the multi-hour solves. Verifier-aligned search infrastructure and search horizon therefore account for more of the gain than model-to-model interaction, with two-family coverage and robustness as smaller, useful additions.

1 Introduction

Formal theorem proving lets a language-model system's output be judged exactly: a Lean 4 proof type-checks against the stated theorem or it does not. The Verified Mechanisms task instantiates this setting with two fixed, off-the-shelf models served through OpenRouter, Qwen 3.5 Flash (fast, inexpensive, February-2026 training cutoff) and GPT-OSS-120B (slower on the accessible provider tier, cheaper per token, strong at high reasoning effort [28], June-2024 cutoff). It asks for a coordination layer that makes the two jointly solve and prove competition mathematics, and for a scientific account of whether the collaboration helps.

We organize the paper around three research questions:

- **RQ1 (scaffold).** How does the controller compare with the kit's raw single-model repair-loop baselines?
- **RQ2 (pairing).** Holding the scaffold fixed, does using both models improve over the better single model, and if so, through what mechanism?
- **RQ3 (hard instances).** What enables solutions on the problems that no short-window configuration solves?

Our design premise is that this task is a coverage problem attached to an exact verifier. The verifier supplies an objective acceptance signal, so no learned or subjective candidate judge is needed, and that shifts value away from dialogue-style coordination, for which compute-matched evidence is weak (§7), and toward candidate diversity, a single consensus decision the verifier cannot check cheaply, and a second set of priors when the first model stalls. What, then, does the second model actually buy? Contributions: (1) a staged, individually-ablatable controller (Fig. 1) with a comparator precheck that aligns search with the true scoring gate; (2) a cap-matched design that isolates pairing effects, floor comparisons against the kit's raw baselines, and a 24-problem custom benchmark with Comparator-validated reference proofs and a held-out split; (3) results for RQ1–RQ3 (§5) with a per-problem complementarity analysis and a benchmark audit (§5.4); (4) ablations and robustness (§6), including a ten-mechanism screening and an unplanned provider-outage stress event.

2 Task and evaluation protocol

Scoring. A problem is solved iff the official *Comparator* accepts every required declaration. The Comparator is a separate scoring program that rebuilds the submitted file from a cold start, checks it with the Lean kernel, and demands kernel-level equality between the submitted statements and the challenge statements under a three-axiom whitelist. Numeric answers must be literal decimals, per-problem spend must not exceed $1.00, and the run must finish within the time cap (8 hours at judging).

The development checker differs from the scoring gate. During search, a fast REPL that keeps Mathlib loaded ("warm") and strips imports checks each candidate; the Comparator builds cold. The two can disagree. A proof the REPL accepts can exceed the Comparator's build-time limit, or its statement can elaborate to a different kernel term under the solution's own imports (§5.4). This gap drives a central design element (§3.3) and a benchmark-validation step (§4).

Budget accounting is fail-closed. If an LLM call fails in a way whose cost cannot be established, the harness permanently stops all further spending for that problem, and the problem scores zero even if a correct proof was saved earlier. The controller therefore runs inside conservative rails: a self-imposed deadline with margin, per-call timeouts, a checkpoint after every improvement, and a degraded mode that finalizes the best saved candidate without further calls once spending is stopped.

Cost structure. Typical per-problem spend is $0.01–$0.10 against the $1.00 cap; the maximum we observed was $0.97, on a budget-exhausted 8-hour run. Wall-clock is usually the binding resource; GPT-OSS latency as served through our provider tier (2–8 minutes per high-effort call, measured) and serial REPL checks dominate it.

Regime	Caps	Problem set	Used for
Raw kit baselines	≈20 min / $1 (both re-run at 30 min post-review)	kit 16	per-model comparison for RQ1; both post-review reruns are cap-matched at 30 min
Cap-matched arms	30 min / $1, 2 seeds	kit 16	RQ1, RQ2: same controller, model set switched
Development evaluations	20 min / $1	custom dev-16	variant selection and mechanism screening (§6)
Held-out checks	20 min / $1	custom held-8	promotion checks only; never used for selection
Long-horizon runs	2–8 h / $1	hard subsets	RQ3; includes validation runs at exact judge settings

Table 1: Evaluation regimes. Every result in §5–§6 is labeled with its regime; comparisons are within-regime unless stated.

Reporting conventions. Repeated identical-configuration runs on the dev-16 vary by up to ±2 problems, driven by three problems whose empirical pass rate across repeated runs is roughly 50%; we therefore report per-problem outcomes wherever a claim depends on them and treat aggregate differences within ±2 as inconclusive. "Cumulative coverage” counts a problem as covered if any run in the stated regime solved it; it is a coverage statement, not a per-run success rate.

3 System

Figure 1 shows the controller: a single anytime escalation ladder whose stages are individually switchable (implementation: `submission/agent.py`, ~1,900 lines over the kit's services). The two models never exchange messages. They interact through shared artifacts — candidate files, error histories, proof skeletons — at a small number of narrow interfaces: pooled sampling (S1), independent answer votes (S0.5), plateau handoff (S3), and alternating sketch duty within decomposition (S4).

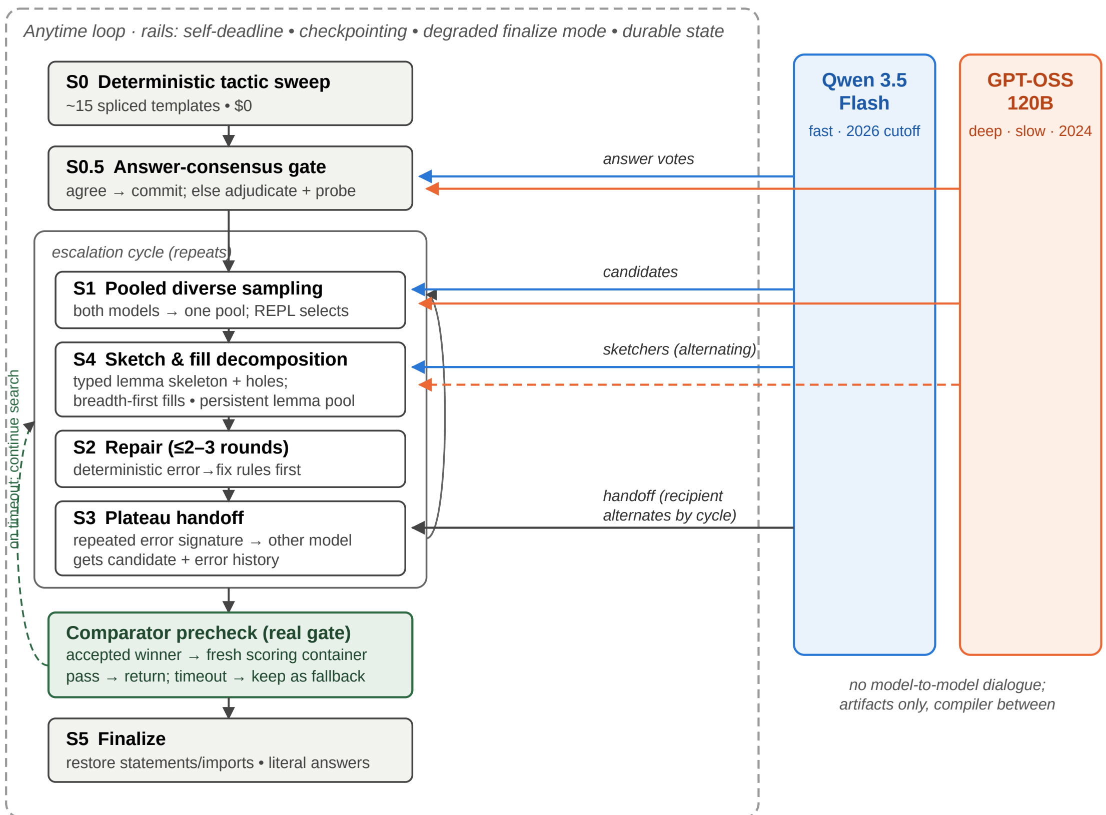

Figure 1: The controller. A staged escalation ladder (left) runs inside the safety rails that fail-closed budget accounting requires; the two models (right) contribute only through artifact-level interfaces. Every REPL-accepted winner must pass a comparator precheck against the real scoring gate before it is returned; a precheck timeout keeps the proof as a fallback while search continues.

3.1 Stages

S0, deterministic sweep ($0). About 15 tactic templates, spliced into the pristine challenge statement, solve the easy third of the kit set before any model is called; since models produce proof bodies and helper lemmas, never statements, the Comparator's statement-equality requirement holds by construction. **S0.5, answer consensus.** For answer-type problems both models independently propose the numeric answer; agreement commits it, disagreement triggers one adjudication round plus inexpensive REPL falsification probes. This is the one decision the verifier cannot check cheaply. **S1, pooled sampling.** Cheap Qwen samples, Qwen samples with reasoning mode enabled (an explicit thinking-token budget; hereafter *Qwen-reasoning*), and GPT-OSS samples enter one deduplicated pool; the REPL selects. **S2, capped repair.** At most 2–3 error-informed rounds with the model that wrote the candidate, after free deterministic error → fix rewrites (option insertions, misnamed-constant substitution). **S3, plateau handoff.** A repeating error fingerprint routes the candidate plus a condensed error history to the other model, motivated by complementary failure modes on the kit baselines (GPT-OSS repeats an identical failed attempt; Qwen's attempts vary without converging) and a 20-month training-cutoff gap. **S4, decomposition.** A sketcher (Qwen-reasoning primary, GPT-OSS alternating) writes an informal solution and a Lean skeleton of standalone, explicitly-typed helper lemmas with `sorry` placeholders; invalid skeleton lines are masked instead of discarded [11]; holes are filled breadth-first (every hole gets a cheap attempt before any gets multi-round repair), and proven lemmas persist in a per-problem pool across re-sketches and restarts. **S5, finalize.** Restore pristine statements and imports, enforce literal answers, re-verify, return.

3.2 Window-aware time constants

All stage and model time constants scale with the available window: a fixed 960-second GPT-OSS timeout had silently disabled decomposition at 20-minute caps until scaled. The outer loop's cycle cap likewise yields to remaining time; an early version stranded three hours of an eight-hour window when a provider outage made all-Qwen cycles fast (§6.3).

3.3 Comparator precheck

REPL-accepted proofs repeatedly exceeded the Comparator's cold-build time limit (three times on one problem, then on others) because kernel-side cost is invisible to the warm checker. The controller therefore runs the same Comparator, in a fresh scoring-style container, on accepted winners before returning them. Acceptance under a wall-clock limit can still depend on machine load, so a precheck timeout does not discard the proof: it is retained as a fallback while search continues for a cheaper one. Section 6.2 quantifies the effect of this component; it involves no second model and is a pure verifier-alignment mechanism. Two further verifier-alignment mechanisms followed from the audit of §5.4 and ship in the submitted agent. Every candidate is composed on the challenge's exact import block (invisible to the import-stripping REPL, decisive for the Comparator), and a precheck rejection on a challenge narrower than `import Mathlib` triggers one repair round confined to core tactics, after which search continues with surface-constrained prompts and linted candidates.

4 Experimental design

Arms. `solo-qwen`, `solo-gptoss`, and `duo` run the identical controller with only the model set switched, at identical caps and seeds, which isolates pairing effects (RQ2) from scaffold effects (RQ1). In the solo arms the cross-model interfaces degrade to same-model equivalents: the answer gate draws two independent samples from the one model, and plateau handoff restarts the same model with fresh context. The arms are *cap-matched, not compute-matched*. Under identical caps the realized effort differs (per 16-problem run, roughly 121–125 LLM calls for the duo, 334–384 for solo-Qwen, and 17–27 for latency-bound solo-GPT-OSS), so RQ2 compares configurations under equal resource limits, not equal consumed compute. The kit's raw baselines supply the scaffold-free floor per model.

Datasets and development boundary. The kit's 16 public problems informed development throughout (failure analysis, calibration); results on them are development-regime results except where the arms protocol makes the comparison internally controlled. To limit overfitting we authored 24 additional problems in the kit's format (number theory, modular arithmetic, divisor counting, factorial valuations, inequalities, telescoping products): 16 form a development set used for variant selection, and 8 were held out and used only to check promotions (two checks during the project; never used for selection). Three deterministic tactic-cascade entries were mined from archived failed subgoals of earlier runs and are therefore development-derived; they are generic tactic patterns, not problem-specific. The agent never observes reference proofs. Both raw single-model loops were also run on all 24 custom problems at 30-minute caps to supply the benchmark's scaffold-free floors (§5.1).

Benchmark validation. Each custom problem was admitted only after its reference proof was machine-checked in the development REPL. Because this paper's own argument is that REPL acceptance does not imply Comparator acceptance, we also ran all 24 reference proofs through the full Comparator gate (fresh scoring container, manifest declaration names, cold build) before submission; the validation script and per-problem results ship with the repository (`scripts/validate_references.py`, `outputs/reference-comparator-validation.json`). All 24 passed (cold-build times 33–225 s, none near the gate's limit), so the benchmark's solvability claims hold under the gate that scores submissions.

Metrics and noise. Primary metric: problems solved under the stated regime's caps, with per-problem outcome grids for any claim that depends on composition. Aggregate differences within the observed ±2 dev-16 band are reported as inconclusive. Where attribution matters (which stage or model produced the winning proof), we use the per-problem event logs shipped with each run.

5 Results

5.1 RQ1: cap-matched reruns show aggregate parity with both raw loops

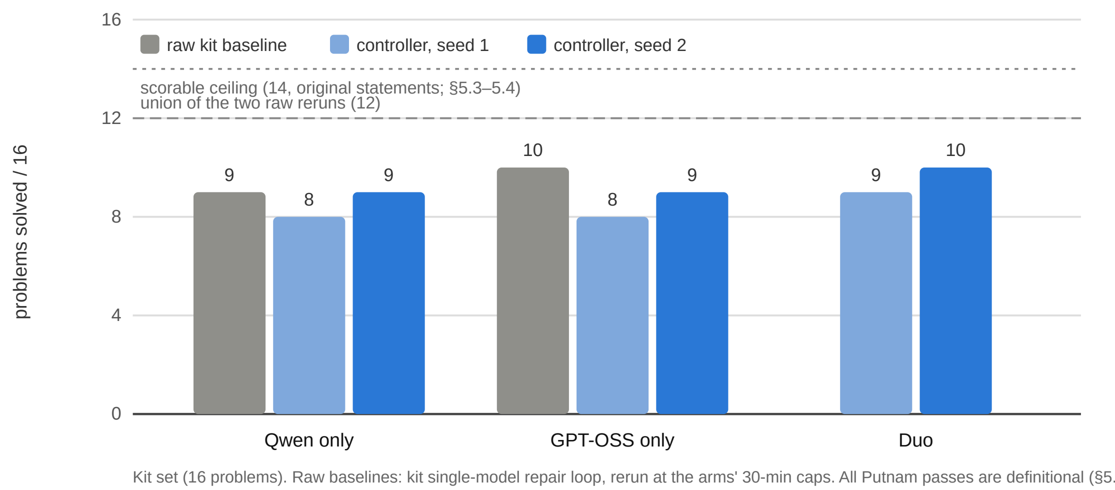

Figure 2: Cap-matched regime: same controller, model set switched, identical 30-minute caps, two seeds per arm. Both raw bars are post-review reruns at the same 30-minute caps (the original ≈20-minute kit baselines scored Qwen 7, GPT-OSS 8). Putnam passes in every bar use the definitional route of §5.4.

The kit's original baselines (Qwen 7, GPT-OSS 8) ran at ≈20-minute caps. Rerun at the arms' own 30-minute caps, the raw Qwen loop scored **9/16**: parity with the controller's solo-Qwen arm (8–9), and its passes included p09, which no controller arm solved at 30 minutes. The raw GPT-OSS loop rerun at the same caps scored **10/16**, at or above its own controller arm (8–9) and level with the duo's better seed, within the ±2 band. It passed the seven easy problems, both Putnams (definitional route, §5.4), and p10, which the controller arms passed once in eight 30-minute attempts; its misses are measured over two clean full windows each (all attempts logged). So the controller's earlier apparent aggregate gain over the raw baselines was mostly wall-clock, not scaffold. Both raw reruns were also cheap ($0.24 and $0.21 per 16-problem run). On the custom benchmark the picture differs. At 30-minute caps the raw Qwen loop scored 12/24 (dev-16 8/16, held-8 4/8) and the raw GPT-OSS loop 9/24 (dev 6/16, held 3/8; union 15/24), whereas the controller reached 11–13/16 on dev-16 at 20-minute caps and, on the held-out 8, 6/8 in both 20-minute checks and 7/8 in a 30-minute rerun (a solo-Qwen arm: 6/8, the same set). On unseen problems the scaffold adds coverage over the raw loop (m03, h06); both miss m05. What survives at matched caps on the kit set: the free S0 sweep covers 6/16 at $0 where the raw loop spends model calls on every problem; the per-problem compositions differ (the raw Qwen rerun missed p05, p10, and putnam_2020_a2, while the controller covers p05 deterministically and reached p10, §5.3); and the controller carries the reliability rails and comparator precheck whose measured value appears at longer horizons (§6.2). In fact the *union* of the two raw reruns covers 12/16, more than any single controller run: cross-family complementarity exists at the raw level, and one controller window realizes only part of it (§5.2).

5.2 RQ2: pairing adds coverage through complementary outcomes; the aggregate gain is small

The duo scored one problem above the best solo arm on both seeds (9 vs. 8; 10 vs. 9) at one-half to one-third of solo-Qwen's cost per run ($0.16–0.17 vs. $0.40–0.45), and in each seed it matched the union of that seed's two solo arms. Because the aggregate difference sits within the ±2 run-to-run band we observed on repeated identical configurations, we do not treat the +1 as strong evidence by itself; and since the two raw loops' union (12/16, §5.1) exceeds the duo's best run (10/16), one window of pairing captures the families' complementarity only partially. The per-problem grid (Table 2) is the more informative result: the solo arms show complementary per-problem *outcomes* across both seeds, and targeted rerun seeds show the headline asymmetries are rate differences, not absolute ones (Table 2 notes). A three-arm replication on the held-out 8 at 30-minute caps under matched light load points the same way: duo 7/8, solo-Qwen 6/8, solo-GPT-OSS 5/8, raw Qwen 4/8. The duo's extra problem (h03, its first solve in four duo runs) came from a Qwen sampling wave in a late cycle (88 Qwen calls, 3 GPT-OSS calls). It is a coverage event under the scaffold, not a cross-model artifact; on these problems the pairing effect is at most one problem and the scaffold effect dominates.

Problem	**Qwen s1/s2**	**GPT-OSS s1/s2**	**Duo s1/s2**	**Note**
7 easy/medium problems	✓✓	✓✓	✓✓	solved by every arm, both seeds (6 of them by the $0 sweep)
p07 (least divisible)	✓✓	✗✗	✓✓	all seeds incl. reruns: solo-GPT-OSS 1/4 (its same-model answer gate committed a wrong path in 3 of 4) vs. 2/2 solo-Qwen and 2/2 duo (two-model gate)
putnam_2018_a1	✗✗	✓✓	✓✓	all seeds incl. reruns: solo-Qwen 2/6, solo-GPT-OSS 3/3, duo 3/3 (duo winners from its own Qwen wave): a rate difference, not a family boundary
putnam_2020_a2	✗✗	✗✓	✗✓	GPT-OSS-side solve in seed 2 for both arms that carry it
p10 (factorial powers)	✗✓	✗✗	✗✗	cured-configuration rerun: duo 0/4, honest misses ($0.02–0.05); across all clean 30-minute attempts p10 passed twice in ten (solo-Qwen seed 2; raw GPT-OSS rerun): a low-probability short-cap solve, not an arm difference
3 remaining provable hard problems	✗✗	✗✗	✗✗	p09, rmo_2000_2, rmo_2001_2: out of reach for every arm at 30 minutes (§5.3)
2 unscorable problems	✗✗	✗✗	✗✗	rmo_2000_3, rmo_2000_6: excluded from the pre-revision scorable ceiling (§5.3, §5.4)

Table 2: Per-problem outcomes, cap-matched regime (✓ = Comparator pass; s1/s2 = seeds). Marks show the original two seeds per arm; the notes report rates including targeted rerun seeds added after review (n up to 6 per cell). Under more seeds, both headline asymmetries become rate differences rather than absolute family boundaries; the duo inherits the favorable side of each. The Putnam rows reflect the original kit statements; every checkmark on them used a definitional route that the upstream statement revision has since eliminated (§5.4).

The answer gate's cross-model value appears as a rate on p07: the duo's two-model gate committed the correct answer in 2/2 runs, while solo-GPT-OSS's same-model gate committed a wrong path in 3 of 4 seeds (Table 2).

Two further RQ2 observations from other regimes, labeled as such. *Robustness (unplanned stress event, long-horizon regime):* during final validation the GPT-OSS provider refused requests for roughly 14 consecutive hours. A solo-GPT-OSS configuration would have scored zero over that span; the duo degraded to Qwen-only behavior and kept solving. *Skeleton diversity (long-horizon regime):* the first full-cap p09 solve grew from a GPT-OSS sketch that Qwen repaired and filled, a structure the solo-Qwen arm did not produce in that run. We report it as a single observed instance, not a systematic effect.

5.3 RQ3: hard instances at short versus long horizons

Six kit problems were unsolved by both original raw baselines. Writing reference proofs for them established that only four are provable as stated. One (rmo_2000_6) is mathematically false as formalized: its second clause asserts a least value of 20 for a set that contains 10. Another (rmo_2000_3) fails to elaborate under the Comparator's actual import environment, because its file imports only two Mathlib modules, which do not bring `Finset.Ico` into scope, a flaw invisible to the import-stripping REPL. The statement itself is true and provable (our full-Mathlib reference proof compiles), but the Comparator's Challenge build fails before any solution is considered; we confirmed in the pinned scoring container that even a correct solution carrying its own `import Mathlib` cannot score. The scorable ceiling is therefore 14/16 under the original statements (certificates in the repository; §5.4 covers the upstream revision that later corrected rmo_2000_6). Table 3 summarizes the record on the provable four.

Problem	Best outcome	Regime and evidence
p09 (IMO 1964)	**Comparator pass; 3 of 3** full-cap runs	8 h/$1 judge settings; $0.10–0.91, 2–7.6 h; each pass required the precheck to reject a costlier accepted proof first (the raw Qwen 30-min rerun also solved p09 once)
p10 (factorial powers)	**Comparator pass**	first solved at 2 h caps ($0.05, 64 min); also solved within a 30-min cap by a solo-Qwen arm (Table 2) and by the cap-matched raw GPT-OSS rerun ($0.025, 24 min), and again under judge-compatible checking ($0.029, 38 min) after the precheck landed
rmo_2000_2	**Comparator pass**	8 h/$1; $0.75; 7.4 h of accumulated lemma-building in one uninterrupted window
rmo_2001_2	unsolved	best attempts (4 h, and a budget-limited ≈12.6 h/$0.89 run with 29 decomposition cycles and 3519 checks) each ended one unfilled hole short

Table 3: Hard-tier record (cumulative coverage; regimes as noted per row). Cumulative coverage across all regimes: 13 of 14 scorable kit problems. The original raw baselines scored 0 on this tier; the cap-matched reruns produced two single-seed passes (raw Qwen p09, raw GPT-OSS p10).

Short-cap solves in this tier are possible but rare: p10 fell within a 30-minute cap under the controller (solo-Qwen arm, seed 2; Table 2) and under the raw GPT-OSS rerun, and the raw Qwen rerun solved p09 once (single seed). The long-horizon pattern is a strong tendency, then, not a law. The systematic record, however, is at long horizons: the controller's p09 solves are 3-for-3 at 8-hour caps (each requiring the precheck), rmo_2000_2 yielded only after 7.4 hours of accumulated lemma-building, and rmo_2001_2 has not yielded at any horizon tried.

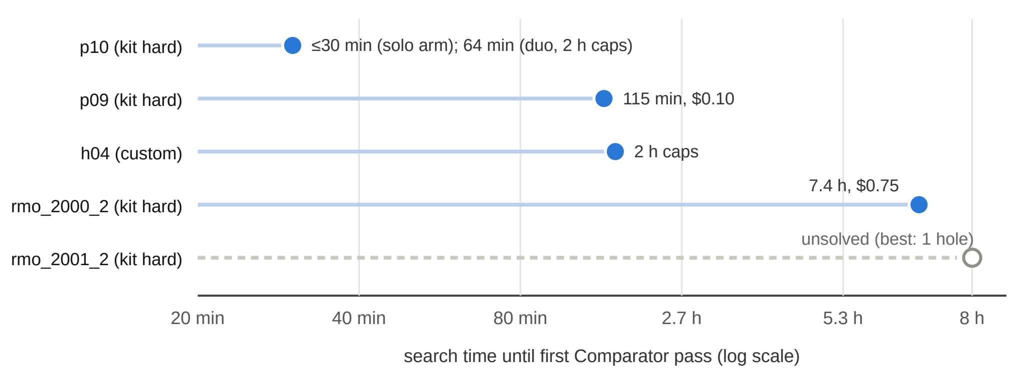

Figure 3: Time until each hard problem first passed the Comparator (hollow marker = still unsolved). p10 falls within short caps (a controller arm; once, the raw GPT-OSS loop); every other solved instance in this tier fell only at a multi-hour horizon under S4 decomposition. The dev-16 aggregate did not change when the short window was doubled from 20 to 40 minutes (11–13 → 11, within the noise band). h04 is a custom problem unsolved by ~19 short-window configurations before the 2-hour run.

Two attribution results temper the collaboration reading of Table 3. First, a *solo-Qwen* arm given the same controller and a 4-hour window also solved p09, through the same trajectory (a costly accepted proof rejected by the precheck, then a cheaper fill that passed), so on that problem the scaffold and horizon carry the outcome, not the pairing. Second, the h04 solve's winning proof had itself been precheck-rejected (the precheck comparator timed out under concurrent machine load) and survived only because rejected winners are retained as fallbacks; on a loaded machine a precheck timeout is not reliable evidence that a proof is genuinely too costly. The 4-hour solo control is shorter than the duo's 8-hour runs, so it bounds them without matching them.

5.4 Upstream statement revisions during the submission window

Late in our submission window the kit's upstream repository revised three challenge statements (PR #9; adopted verbatim here, and its tests pass). Both Putnam problems now state their answers as explicit closed forms. Originally the answer was a solver-supplied definition, and an audit of our event logs found that *every* historical Putnam pass in this paper (every arm and seed, both raw reruns) instantiated it with the defining condition itself: legal under the original statements, but shortcut discovery that does not transfer. Separately, rmo_2000_6's second bound was corrected from 20 to 10, precisely the witness our falsity certificate exhibits (§5.3; $5^3 \cdot 2^4 = 2000$, $a \cdot b = 10$), which raises the revised-statement ceiling to 15 of 16 (rmo_2000_3 is unchanged). Post-revision runs confirm this. Across twenty-four problem-attempts (three arms at 30 min, two seeds; a 2-hour and an 8-hour duo run) no arm scored on any revised problem, and the Putnams now behave as genuine hard-tier instances (the 8-hour putnam_2020_a2 attempt was our first to exhaust its $1.00 budget). The 8-hour rmo_2000_6 attempt is the most telling single result: within 75 minutes the duo found a REPL-accepted proof of the corrected statement, and its comparator precheck rejected it in-run; final scoring confirms that both files build but kernel-level statement comparison fails. Import-ablation probes pin the mechanism. Under the challenge's exact two imports the statement matches (a `sorry` probe fails only on the sorry axiom); adding `Mathlib.Tactic`, which the proof's tactics need, or all of Mathlib elaborates the same text to a different kernel term. The gap documented on the challenge side (rmo_2000_3) thus also binds on the solution side. It is nevertheless scorable: a reference proof inside the challenge's import surface (core tactics; `decide`-driven bounded exhaustion in place of `interval_cases`) passes the Comparator in 14 s, which certifies that ceiling. We then validated the two mechanisms this motivated (§3.3). A live rerun fired the whole path (precheck rejection at 12 min, lint, confined round), but six fresh core-only attempts per prompt revision, each scored by the Comparator, passed 0/6 both before and after we revised the technique guidance; neither model reliably finds the bounded-`decide` structure our reference uses. The fallback is bounded by model capability under a restricted surface, not by plumbing. The composition rule is the dependable gain.

6 Ablations and robustness

6.1 Mechanism screening on the development set

An initial selection loop promoted two changes (window-proportional time constants and breadth-first hole filling), raising the dev-16 from 11 to 13 with the held-out set stable at 6/8 on both checks. We then screened ten further mechanisms drawn from the literature and from our own failure analysis (Fig. 4). None of the ten reproducibly improved on the tuned default. The only score above the band (verified premise hints, 13/16, with every ≈50%-pass-rate problem landing favorably) fell to 10/16 on its repeat seed, and two mechanisms' triggers fired at most once. The screening is single-toggle and short-window (no interactions, no other horizons). One pattern within it is consistent: every mechanism that lengthened prompts with guidance material scored at or below the default while costing more. At short windows this system gains more from additional cycles and sample diversity than from added instruction.

6.2 Precheck and reliability

The comparator precheck's contribution is estimated by counterfactual replay of run logs: under a no-precheck return policy one full-cap validation run would have returned two accepted-but-costly proofs and scored 0/4; with the precheck both were rejected internally and a later proof passed. Before/after evidence agrees: a problem lost three times to scoring-time build timeouts was solved on the first post-precheck attempt. The h04 case (§5.3) shows the retain-don't-discard half mattering in the opposite direction. Across the final configuration's runs (container restarts, the 14-hour provider outage, out-of-memory REPL failures) no run ended with unaccounted cost, and every full-cap problem attempt (sixteen to date) is traceable in the shipped event logs to a scored outcome or a documented environmental cause.

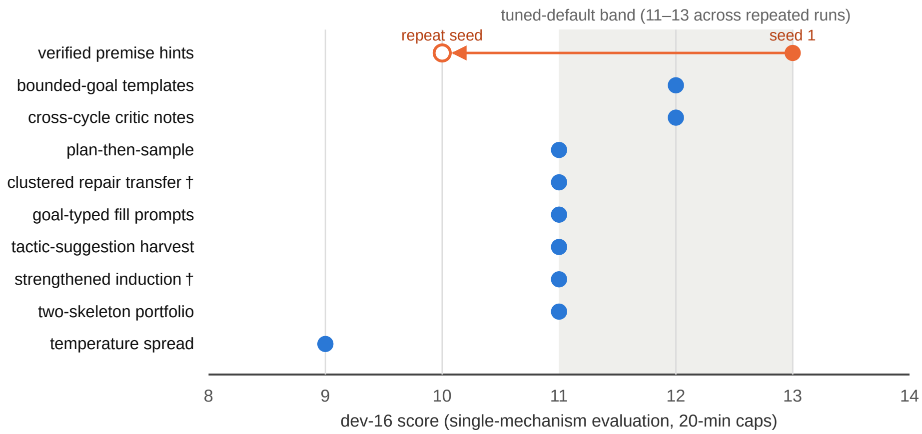

Figure 4: Mechanism screening (development regime). Ten mechanisms, each behind an independent flag, each evaluated with only that flag enabled. † = the mechanism's trigger condition fired at most once during its run, so its score is uninformative about the mechanism itself.

6.3 Negative results retained

Deeper per-hole reasoning budgets at short windows: no additional structural solves, +16% cost. GPT-OSS inside short-window sampling waves: its latency halves completed cycles. The cycle-cap interaction of §3.2 (an 8-hour window ended at 4.9 hours) is fixed in the submitted configuration.

7 Related work

Repeated sampling against a verifier converts coverage into accuracy [1], and is the baseline any richer mechanism must beat at matched cost. Cross-family candidate pooling captures most ensemble benefit with two models [2], and pays when the selector is strong [3]; in our setting final proof validity is decided mechanically by the Comparator. Compiler-feedback repair helps most when shallow [4, 5, 6, 7]; equal-budget self-repair can lose to resampling [8], and models correct located errors better than they locate them [9], which motivates handing a candidate plus located errors to a *different* model instead of critiquing. Sketch-then-fill decomposition underlies the strongest current systems [10–15], and agentic loops around unmodified general models are now competitive with fine-tuned provers [7, 23, 24]; GPT-OSS-120B appears as the informal reasoner of an open hard-mode pipeline [25] and among the most cost-efficient provers in [26]. Verifier-gated cascades [16] and self-consistency [17] supply the cost-allocation and answer-commitment patterns we adopt. We did not build dialogue-style coordination: compute-matched evaluations of multi-agent debate are largely negative [18, 19, 20, 21], and mixture-of-agents synthesis underperforms selection [22, 3]. Benchmarks: miniF2F [29] and PutnamBench [30] define the formal-competition setting; the kit's Putnam items follow PutnamBench's solver-supplied-answer convention, whose definitional-instantiation loophole §5.4 documents. AlphaProof [31] marks the frontier with a trained RL prover; we instead coordinate two fixed, unmodified models. Our evaluation methodology follows [27].

8 Limitations

Sample sizes are small (16 kit problems, 24 custom) and the dev-16 varies by ±2 across identical runs, so we avoid significance claims and report per-problem outcomes; the RQ2 aggregate difference (+1 on both seeds) is within that band and the complementarity pattern of Table 2 is the sturdier finding. The long-horizon solo control (4 h) is shorter than the duo's full-cap runs (8 h). The original raw baselines ran at ≈20-minute caps; post-review cap-matched reruns close this for both models, and single-seed results (the raw p09 and p10 solves) are reported as such. All 24 custom challenges import full Mathlib, so the custom benchmark cannot exhibit the import-surface gap of §5.4, and the long-horizon evidence on the revised statements is one seed. Development-environment conditions (provider outages, container restarts, memory pressure, shared-machine saturation; a duo held-8 run under saturation scored 4/8 and one raw GPT-OSS custom run lost ten problems to REPL timeouts, and both are reported and were rerun under light load) affected several runs; every incident is documented per-run, and the one development-only mitigation is inactive under judging defaults. Finally, these are two fixed mid-scale models: the demonstrated range (full easy/medium tier, several hard instances, olympiad-final tier unsolved) is consistent with published results for comparable backbones [6, 14, 25, 26].

9 Conclusion

When proof correctness is externally verifiable, most of the value of a multi-LLM theorem-proving system in our setting lay in verifier-aligned search infrastructure and complementary candidate coverage, not in conversational coordination. Cap-matched baseline reruns tempered the headline. Both raw loops reach aggregate parity with the controller's arms at 30 minutes, which relocates the controller's demonstrated value to zero-cost deterministic coverage, per-problem complementarity, robustness, its verifier-alignment machinery, and, on the held-out custom problems, where the raw loop scores 4/8 against the scaffolded arms' 6–7/8, search organization itself. The same reference-proof discipline doubled as a benchmark audit; an upstream revision later corrected one defective statement exactly as our falsity certificate prescribed (§5.4). Two model families added smaller but real benefits: per-problem coverage the aggregate understates, one consensus decision with a concrete case (p07), an observed cross-family proof structure, and robustness to a provider outage that would have zeroed a single-model configuration. Among the remaining structural instances, systematic success appeared only at long horizons under decomposition; none fell to the short-window prompting variants we screened. Within the mechanisms and budgets we tested, that ordering — controller, then coverage, then horizon — is the paper's main empirical claim, and no model-to-model dialogue was needed to reach it.

Reproducibility and disclosure. The repository contains the controller, the custom benchmark with Comparator-validated reference proofs, per-run artifacts and event logs for every result above, and `RUNBOOK.md`; `scripts/judge_check.sh` passes on the submitted configuration. Appendices: `RESEARCH.md`, `PART2.md`, `EXPERIMENTS.md`, `RESEARCH_LOOP.md`. This submission was developed with substantial assistance from Claude (Anthropic): code, experiments, analysis, and writing, as disclosed in the submission form.

References

[1] B. Brown et al. Large Language Monkeys: Scaling Inference Compute with Repeated Sampling. arXiv:2407.21787, 2024.
[2] F. Vallecillos-Ruiz et al. Wisdom and Delusion of LLM Ensembles for Code Generation and Repair. arXiv:2510.21513, 2025.
[3] A. Maryanskyy et al. When Agents Disagree: The Selection Bottleneck in Multi-Agent LLM Pipelines. arXiv:2603.20324, 2026.
[4] H. Wang et al. Kimina-Prover Preview: Towards Large Formal Reasoning Models with Reinforcement Learning. arXiv:2504.11354, 2025.
[5] Y. Lin et al. Goedel-Prover-V2: Scaling Formal Theorem Proving with Scaffolded Data Synthesis and Self-Correction. arXiv:2508.03613, 2025.
[6] A. Ospanov et al. APOLLO: Automated LLM and Lean Collaboration for Advanced Formal Reasoning. arXiv:2505.05758, 2025.
[7] Y. Zhou et al. Solving Formal Math Problems by Decomposition and Iterative Reflection. arXiv:2507.15225, 2025.
[8] T. X. Olausson et al. Is Self-Repair a Silver Bullet for Code Generation? arXiv:2306.09896, 2023.
[9] G. Tyen et al. LLMs cannot find reasoning errors, but can correct them given the error location. arXiv:2311.08516, 2023.
[10] A. Q. Jiang et al. Draft, Sketch, and Prove: Guiding Formal Theorem Provers with Informal Proofs. arXiv:2210.12283, 2022.
[11] C. Cao et al. Reviving DSP for Advanced Theorem Proving in the Era of Reasoning Models. arXiv:2506.11487, 2025.
[12] Z. Z. Ren et al. DeepSeek-Prover-V2: Advancing Formal Mathematical Reasoning via Reinforcement Learning for Subgoal Decomposition. arXiv:2504.21801, 2025.
[13] S. Varambally et al. Hilbert: Recursively Building Formal Proofs with Informal Reasoning. arXiv:2509.22819, 2025.
[14] K. Baba et al. Prover Agent: An Agent-Based Framework for Formal Mathematical Proofs. arXiv:2506.19923, 2025.
[15] L. Chen et al. Seed-Prover: Deep and Broad Reasoning for Automated Theorem Proving. arXiv:2507.23726, 2025.
[16] L. Chen, M. Zaharia, and J. Zou. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. arXiv:2305.05176, 2023.
[17] X. Wang et al. Self-Consistency Improves Chain of Thought Reasoning in Language Models. arXiv:2203.11171, 2022.
[18] J. Huang et al. Large Language Models Cannot Self-Correct Reasoning Yet. arXiv:2310.01798, 2023.
[19] A. Smit et al. Should we be going MAD? A Look at Multi-Agent Debate Strategies for LLMs. arXiv:2311.17371, 2023.
[20] H. Zhang et al. Stop Overvaluing Multi-Agent Debate — We Must Rethink Evaluation and Embrace Model Heterogeneity. arXiv:2502.08788, 2025.
[21] L. B. Kaesberg et al. Voting or Consensus? Decision-Making in Multi-Agent Debate. arXiv:2502.19130, 2025.
[22] W. Li et al. Rethinking Mixture-of-Agents: Is Mixing Different Large Language Models Beneficial? arXiv:2502.00674, 2025.
[23] B. Requena et al. A Minimal Agent for Automated Theorem Proving. arXiv:2602.24273, 2026.
[24] P.-N. Kung et al. LEAP: Supercharging LLMs for Formal Mathematics with Agentic Frameworks. arXiv:2606.03303, 2026.
[25] C. Liu et al. Discover and Prove: An Open-source Agentic Framework for Hard Mode Automated Theorem Proving in Lean 4. arXiv:2604.15839, 2026.
[26] T. Klingner et al. Evaluation of LLMs for Mathematical Formalization in Lean. arXiv:2606.05632, 2026.
[27] S. Kapoor et al. AI Agents That Matter. arXiv:2407.01502, 2024.
[28] OpenAI. gpt-oss-120b & gpt-oss-20b Model Card. arXiv:2508.10925, 2025.
[29] K. Zheng, J. M. Han, and S. Polu. miniF2F: a cross-system benchmark for formal Olympiad-level mathematics. arXiv:2109.00110, 2021.
[30] G. Tsoukalas et al. PutnamBench: Evaluating Neural Theorem-Provers on the Putnam Mathematical Competition. arXiv:2407.11214, 2024.
[31] T. Hubert et al. Olympiad-level formal mathematical reasoning with reinforcement learning. Nature, 2025.